Security Review Report
NM-0411-0922 - World V2 Contracts for World Id Registry

NETHERMIND
SECURITY

(May 27, 2026)

Contents

1 Executive Summary

This document presents the results of a security review conducted by Nethermind Security for the World ID Registry and Verifier V2 contracts.The World ID protocol implements a privacy-preserving identity management system. It leverages zero-knowledge cryptography to allow individuals to prove claims about themselves without disclosing personal identifiers. The system's on-chain anchor relies on a registry contract to manage user accounts and their authorized authenticators, paired with a verifier contract that validates the zero-knowledge proofs produced by those accounts.

The scope of this audit focuses on the protocol's V2 upgrade, which builds upon the existing architecture to introduce the following key improvements:

- **Enhanced Account Management (Registry):** Introduces distinct authenticator classes (Admin and Proving) to safeguard account control, ensuring accounts always maintain a manageable state.
- **Streamlined Recovery (Registry):** Simplifies the account recovery process into a single-step update. This is protected by a time-locked revert window, providing a fail-safe that allows legitimate users to undo unauthorized or malicious changes.
- **Proof Validation (Verifier):** The verifier was updated to add a restriction on the action payload.

The audit comprises 1310 lines of Solidity code. **The audit was performed using** (a) manual analysis of the codebase, and (b) automated analysis tools.

Along this document, we report two points of attention, where one is classified as `Low`, and one is classified as `Informational` severity. The issues are summarized in Fig. 1.

This document is organized as follows. Section 2 presents the files in the scope. Section 3 presents the summary of issues. Section 4 presents the System Overview. Section 5 discusses the risk rating methodology. Section 6 details the issues. Section 7 discusses the documentation provided by the client for this audit. Section 8 presents the test suite evaluation and automated tools used. Section 9 concludes the document.

Fig. 1: Distribution of issues: Critical (0), **High** (0), **Medium** (0), **Low** (1), **Undetermined** (0), **Informational** (1), **Best Practices** (0). **Distribution of status: Fixed** (1), **Acknowledged** (1), **Mitigated** (0), **Unresolved** (0)

Summary of the Audit

Audit Type	Security Review
Initial Report	May 22, 2026
Final Report	May 27, 2026
Initial Commit	4fa80cb
Final Commit	ace6447
Documentation Assessment	High
Test Suite Assessment	High

2 Audited Files

	Contract	LoC	Comments	Ratio	Blank	Total
1	core/WorldIDVerifierV2Unreleased.sol	61	20	32.8%	7	88
2	core/WorldIDRegistry.sol	616	161	26.1%	130	907
3	core/WorldIDVerifier.sol	197	60	30.5%	38	295
4	core/WorldIDVerifierV2.sol	33	17	51.5%	4	54
5	core/WorldIDRegistryV2Unreleased.sol	403	115	28.5%	73	591
	Total	**1310**	**373**	**28.5%**	**252**	**1935**

3 Summary of Issues

	Finding	Severity	Update
1	Bitmap collision for legacy pubkeyId >= 48	Low	Acknowledged
2	Zero root recorded as valid on fresh V2 deployment	Info	Fixed

4 System Overview

The World ID protocol implements a privacy-preserving identity management system utilizing zero-knowledge cryptography. **This audit focuses on the protocol's V2 upgrade.** The updated system transitions from its V1 architecture to V2 by introducing new authenticator classes, streamlining the recovery agent flow, and improving root validity tracking.

4.1 WorldIDRegistryV2Unreleased

The V2 registry inherits from `WorldIDRegistry` and implements `IWorldIDRegistryV2`. It introduces new errors, a `PreviousRecoveryAgentUpdate` struct, and revamped recovery-agent management. While reusing the V1 storage layout for accounts, authenticators, and roots, V2 introduces two state mappings:

```
mapping(uint256 => uint256) internal _rootToValidityTimestamp;
mapping(uint256 => PreviousRecoveryAgentUpdate) internal _prevRecoveryAgentUpdates;
```

- **`_rootToValidityTimestamp`**: Records the exact timestamp at which a root ceases to be the latest.
- **`_prevRecoveryAgentUpdates`**: Retains the previous Recovery Agent during an active revert window.

4.1.1 Root Validity Update

The root validity mechanism was updated to ensure that a root's Time-to-Live (TTL) window begins strictly from the moment it is *superseded* by a new root, rather than from its initial creation time. To facilitate this, the V2 system utilizes the `_rootToValidityTimestamp` mapping to capture the exact replacement timestamp via an overridden `_recordCurrentRoot` function. Furthermore, a new view function, `getRootExpiration`, has been introduced to expose the precise expiration timestamp of any previously recorded root.

4.1.2 Authenticator Classes

Authenticators are entities authorized to perform operations on behalf of a user's World ID. V2 introduces two distinct authenticator classes that can coexist on the same account:

- **Admin Authenticators:** Possess an on-chain management address. They are granted administrative rights to manage the account and handle other authenticators.
- **Proving Authenticators:** Restricted purely to proof generation without an associated on-chain management address. To prevent these authenticators from being permanently orphaned, a new system invariant prevents the removal of the last Admin Authenticator if Proving Authenticators still exist on the account.

To support both classes within the existing V1 storage, the 96-bit pubkey bitmap is split. Bits 0–47 encode `pubkeyId` occupancy, and bits 48–95 encode the authenticator class. The hard cap on authenticators per account is correspondingly halved to `MAX_AUTHENTICATORS_V2_HARD_LIMIT = 48` (enforced by `setMaxAuthenticators`).

The following entry points were modified to handle both classes:

- **`insertAuthenticator`**: Now accepts `address(0)` to insert a Proving authenticator by setting the class flag in the bitmap without writing a global mapping entry. Non-zero addresses follow the standard V1 Admin path.
- **`removeAuthenticator`**: Reads the class flag to dictate the removal path. For Proving authenticators, the provided address must be `address(0)`. For Admin authenticators, it deletes the mapping entry, ensuring the account remains manageable if Proving authenticators still exist.
- **`updateAuthenticator`**: Removed in V2. The function now always reverts with `MethodUnsupported`.

4.1.3 Recovery Agent Updates

The upgrade replaced the three-step flow with a single-step update protected by a revert window. During a revert window, the active agent is determined by the internal helper `_getEffectiveRecoveryAgent`:

$$\text{effective agent} = \begin{cases} \texttt{prev.prevRecoveryAgent} & \text{if } \texttt{prev.invalidAfter} \neq 0 \text{ and } \texttt{block.timestamp} < \texttt{prev.invalidAfter} \\ \texttt{\_getRecoveryAgent(leafIndex)} & \text{otherwise} \end{cases} \tag{1}$$

The following entry points were added to implement the update:

- **`updateRecoveryAgent`**: Applies the new Recovery Agent immediately on-chain. It records the previous agent alongside an `invalidAfter` timestamp (current time + cooldown). Reverts with `RecoveryAgentUpdateStillActive` if a prior update is still within its revert window.
- **`revertRecoveryAgentUpdate`**: Restores the previous agent on-chain and clears the entry. Reverts with `NoActiveRecoveryAgentUpdate` or `RecoveryAgentUpdateWindowExpired` depending on the state of the revert window.

Note: Legacy V1 entry points for initiating, executing, and canceling updates are overridden to revert with `MethodUnsupported`.

4.2 WorldIDVerifierV2Unreleased

The V2 verifier inherits from `WorldIDVerifier`, implementing strict prefix-byte checks to ensure actions intended for one proof type cannot be maliciously replayed as another.

The contract exposes two key entry points:

- **`verify`**: Validates a Uniqueness Proof. It strictly enforces that the top byte of the action (`action » 248`) is `0` (the prefix reserved for non-session actions).
- **`verifySession`**: Validates a Session Proof. It reads the action from `sessionNullifier[1]` and enforces that its top byte equals `2` (the session prefix).

5 Risk Rating Methodology

The risk rating methodology used by Nethermind Security follows the principles established by the OWASP Foundation. The severity of each finding is determined by two factors: **Likelihood** and **Impact**.

Likelihood measures how likely the finding is to be uncovered and exploited by an attacker. This factor will be one of the following values:

a) **High**: The issue is trivial to exploit and has no specific conditions that need to be met;

b) **Medium**: The issue is moderately complex and may have some conditions that need to be met;

c) **Low**: The issue is very complex and requires very specific conditions to be met.

When defining the likelihood of a finding, other factors are also considered. These can include but are not limited to motive, opportunity, exploit accessibility, ease of discovery, and ease of exploit.

Impact is a measure of the damage that may be caused if an attacker exploits the finding. This factor will be one of the following values:

a) **High**: The issue can cause significant damage, such as loss of funds or the protocol entering an unrecoverable state;

b) **Medium**: The issue can cause moderate damage, such as impacts that only affect a small group of users or only a particular part of the protocol;

c) **Low**: The issue can cause little to no damage, such as bugs that are easily recoverable or cause unexpected interactions that cause minor inconveniences.

When defining the impact of a finding, other factors are also considered. These can include but are not limited to Data/state integrity, loss of availability, financial loss, and reputation damage. After defining the likelihood and impact of an issue, the severity can be determined according to the table below.

		Severity Risk		
Impact	**High**	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Info/Best Practices	Low	Medium
	Undetermined	Undetermined	Undetermined	Undetermined
		Low	**Medium**	**High**
		Likelihood		

To address issues that do not fit a High/Medium/Low severity, Nethermind Security also uses three more finding severities: **Informational**, **Best Practices**, and **Undetermined**.

a) **Informational** findings do not pose any risk to the application, but they carry some information that the audit team intends to pass to the client formally;

b) **Best Practice** findings are used when some piece of code does not conform with smart contract development best practices;

c) **Undetermined** findings are used when we cannot predict the impact or likelihood of the issue.

6 Issues

6.1 [Low] Bitmap collision for legacy pubkeyId >= 48

File(s): `WorldIDRegistryV2Unreleased.sol`

Description: `WorldIDRegistryV2` splits the 96-bit pubkey bitmap into two halves: bits [0-47] encode occupancy and bits [48-95] encode the authenticator class (Proving vs Management). The hard limit `MAX_AUTHENTICATORS_V2_HARD_LIMIT` is therefore lowered to 48.

However, V1 (`WorldIDRegistry`) exposed `setMaxAuthenticators` with the hard limit of 96. Consequently, any V1 account that registered an authenticator at a `pubkeyId >= 48` will have its occupancy bit located in the half of the bitmap that V2 now interprets as authenticator-class flags. Post-upgrade, those accounts will be silently treated as having one or more Proving Authenticators that never actually existed, corrupting the bitmap and breaking removal and class-mismatch invariants throughout V2.

Recommendation(s): Before the upgrade, consider an off-chain verification that no existing account has an authenticator registered with pubkeyId >= 48. As a defensive measure, also lower `_maxAuthenticators` to 48 prior to upgrading.

Status: Acknowledged

Update from the client: Acknowledged, currently the maximum number of authenticators gloablly is 7, but it definitely something we plan to take into account before performing the v2 upgrade.

6.2 [Info] Zero root recorded as valid on fresh V2 deployment

File(s): `WorldIDRegistry.sol`

Description: In the case of a fresh deployment, the `initialize` function calls `_recordCurrentRoot()` after inserting the sentinel leaf. In V2, the overridden version of this function first captures `_latestRoot` (which defaults to 0 on a fresh deployment) and writes `_rootToValidityTimestamp[0] = block.timestamp` before delegating to `super._recordCurrentRoot()`.

```
function initialize(...)
    public
    virtual
    initializer
{
    __BaseUpgradeable_init(EIP712_NAME, EIP712_VERSION, feeRecipient, feeToken, registrationFee);

    _treeDepth = initialTreeDepth;
    _tree.initWithDefaultZeroes(_treeDepth);

    _tree.insert(uint256(0));
    _nextLeafIndex = 1;
    // @audit `_recordCurrentRoot` result in marking the 0 root as valid.
    _recordCurrentRoot();

    _maxAuthenticators = 7;
    _rootValidityWindow = 3600;
    _recoveryAgentUpdateCooldown = 14 days;
}
```

Consequently, the zero root is recorded as a known, valid root that remains valid for the duration of the `_rootValidityWindow`. Any zero-knowledge proof verified against a root of 0 during this initial window will be incorrectly accepted by `isValidRoot(0)`.

Recommendation(s): To ensure robustness for any future fresh deployments, consider updating the `_recordCurrentRoot()` or `isValidRoot()` functions to avoid recording the `_latestRoot` as valid if it is 0.

Status: Fixed

Update from the client: Acknowledged and fixed at commit hash 08c2fbf7e84348253b4f4870f2675066d440e2c2, though likely not a practical concern we should guard this for future deployments.

7 Documentation Evaluation

Software documentation refers to the written or visual information that describes the functionality, architecture, design, and implementation of software. It provides a comprehensive overview of the software system and helps users, developers, and stakeholders understand how the software works, how to use it, and how to maintain it. Software documentation can take different forms, such as user manuals, system manuals, technical specifications, requirements documents, design documents, and code comments. Software documentation is critical in software development, enabling effective communication between developers, testers, users, and other stakeholders. It helps to ensure that everyone involved in the development process has a shared understanding of the software system and its functionality. Moreover, software documentation can improve software maintenance by providing a clear and complete understanding of the software system, making it easier for developers to maintain, modify, and update the software over time. Smart contracts can use various types of software documentation. Some of the most common types include:

- Technical whitepaper: A technical whitepaper is a comprehensive document describing the smart contract's design and technical details. It includes information about the purpose of the contract, its architecture, its components, and how they interact with each other;
- User manual: A user manual is a document that provides information about how to use the smart contract. It includes step-by-step instructions on how to perform various tasks and explains the different features and functionalities of the contract;
- Code documentation: Code documentation is a document that provides details about the code of the smart contract. It includes information about the functions, variables, and classes used in the code, as well as explanations of how they work;
- API documentation: API documentation is a document that provides information about the API (Application Programming Interface) of the smart contract. It includes details about the methods, parameters, and responses that can be used to interact with the contract;
- Testing documentation: Testing documentation is a document that provides information about how the smart contract was tested. It includes details about the test cases that were used, the results of the tests, and any issues that were identified during testing;
- Audit documentation: Audit documentation includes reports, notes, and other materials related to the security audit of the smart contract. This type of documentation is critical in ensuring that the smart contract is secure and free from vulnerabilities.

These types of documentation are essential for smart contract development and maintenance. They help ensure that the contract is properly designed, implemented, and tested, and they provide a reference for developers who need to modify or maintain the contract in the future.

Remarks about World's V2 contracts documentation

The World team provided pull requests with detailed descriptions of the changes introduced in this V2 upgrade, which were instrumental in understanding the scope and intent of each modification. The team also provided a general overview of the system during the kick-off call, with a specific focus on the V2 changes.

Moreover, the World team remained highly proactive and available throughout the engagement, promptly addressing all questions and concerns raised by the Nethermind Security team.

8 Test Suite Evaluation

8.1 Tests Output

```
> forge test

Ran 6 tests for test/core/CredentialSchemaIssuerRegistryUpgrade.t.sol:CredentialSchemaIssuerRegistryUpgradeTest
[PASS] test_CannotInitializeTwice() (gas: 75200)
[PASS] test_ImplementationCannotBeInitialized() (gas: 2856342)
[PASS] test_OwnerCannotRegisterWithoutUpgrade() (gas: 92875)
[PASS] test_OwnershipTransfer() (gas: 2875111)
[PASS] test_UpgradeFailsForNonOwner() (gas: 2849072)
[PASS] test_UpgradeSuccess() (gas: 3045157)
Suite result: ok. 6 passed; 0 failed; 0 skipped; finished in 16.74ms (5.96ms CPU time)

Ran 11 tests for test/crosschain/Gateway.t.sol:GatewayTest
[PASS] test_bothGateways_sequentialRelay() (gas: 4329898)
[PASS] test_ethereumMPTGateway_e2e() (gas: 3223588)
[PASS] test_ethereumMPTGateway_revert_unsupportedAttribute() (gas: 2887115)
[PASS] test_ethereumMPTGateway_supportsAttribute() (gas: 2494199)
[PASS] test_permissionedGateway_e2e() (gas: 1576421)
[PASS] test_permissionedGateway_revert_emptyPayload() (gas: 901723)
[PASS] test_permissionedGateway_revert_nonOwner() (gas: 1243464)
[PASS] test_permissionedGateway_revert_unauthorizedGateway() (gas: 1242075)
[PASS] test_permissionedGateway_revert_wrongChainHead() (gas: 1299603)
[PASS] test_permissionedGateway_revert_wrongRecipient() (gas: 1205726)
[PASS] test_permissionedGateway_supportsAttribute() (gas: 896187)
Suite result: ok. 11 passed; 0 failed; 0 skipped; finished in 22.18ms (5.47ms CPU time)

Ran 1 test for test/core/WorldIDRegistryV2.t.sol:WorldIDRegistryV2WIP102OrphanTest
[PASS] test_V1PendingUpdateOrphanedAfterUpgrade() (gas: 11717279)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 24.47ms (15.37ms CPU time)

Ran 5 tests for test/crosschain/E2E.t.sol:E2ETest
[PASS] test_e2e_fullPipeline() (gas: 899833)
[PASS] test_e2e_multipleKeys() (gas: 1335581)
[PASS] test_e2e_nothingChanged_reverts() (gas: 698254)
[PASS] test_e2e_sequentialUpdates_chainExtends() (gas: 654583)
[PASS] test_e2e_unknownRootInvalid() (gas: 26457)
Suite result: ok. 5 passed; 0 failed; 0 skipped; finished in 6.09ms (3.98ms CPU time)

Ran 12 tests for test/crosschain/Attributes.t.sol:AttributesTest
[PASS] testFuzz_split_roundTrip(bytes4,bytes) (runs: 256, : 4857, ~: 4840)
[PASS] testFuzz_split_selectorPreserved(bytes4) (runs: 256, : 1108, ~: 1108)
[PASS] test_compoundSelectorValues() (gas: 448)
[PASS] test_compoundSelectorsAreDistinct() (gas: 2112)
[PASS] test_split_bytes32() (gas: 1268)
[PASS] test_split_disputeGame() (gas: 6507)
[PASS] test_split_dynamicBytesArrays() (gas: 11190)
[PASS] test_split_exactlyFourBytes() (gas: 1234)
[PASS] test_split_l1MptProof() (gas: 13306)
[PASS] test_split_revertsEmpty() (gas: 6857)
[PASS] test_split_revertsTooShort() (gas: 7011)
[PASS] test_split_twoDynamicBytes() (gas: 7104)
Suite result: ok. 12 passed; 0 failed; 0 skipped; finished in 30.49ms (21.38ms CPU time)

Ran 3 tests for test/core/WorldIDRegistryV2RaceCondition.t.sol:WorldIDRegistryV2RaceConditionTest
[PASS] test_V1_RootInvalidImmediatelyOnceReplacedAfterTTL() (gas: 911586)
[PASS] test_V2_RootRemainsValidAfterReplacementWhenTTLExpired() (gas: 5597752)
[PASS] test_getRootExpiration() (gas: 5602003)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 29.69ms (24.36ms CPU time)

Ran 15 tests for test/core/WorldIDVerifierTest.t.sol:ProofVerifier
[PASS] test_CannotUpdateOprfKeyRegistryToZeroAddress() (gas: 18263)
[PASS] test_ExpiresAtTooOld() (gas: 58438)
[PASS] test_InvalidRoot() (gas: 45816)
[PASS] test_OnlyOwnerCanUpdateOprfKeyRegistry() (gas: 145897)
[PASS] test_SessionExpiresAtTooOld() (gas: 60882)
[PASS] test_SessionInvalidRoot() (gas: 47578)
[PASS] test_SessionSuccess() (gas: 356081)
[PASS] test_SessionWrongCredentialIssuer() (gas: 356474)
[PASS] test_SessionWrongProof() (gas: 346482)
```

```
[PASS] test_SessionWrongRpId() (gas: 356584)
[PASS] test_Success() (gas: 353551)
[PASS] test_UpdateOprfKeyRegistry() (gas: 151010)
[PASS] test_WrongCredentialIssuer() (gas: 354435)
[PASS] test_WrongProof() (gas: 343652)
[PASS] test_WrongRpId() (gas: 354391)
Suite result: ok. 15 passed; 0 failed; 0 skipped; finished in 79.10ms (77.47ms CPU time)

Ran 9 tests for test/core/WorldIDVerifierUpgrade.sol:VerifierUpgradeTest
[PASS] test_CannotInitializeTwice() (gas: 24886)
[PASS] test_ImplementationCannotBeInitialized() (gas: 2504199)
[PASS] test_OnlyOwnerCanUpdate() (gas: 35204)
[PASS] test_OwnershipTransfer() (gas: 2572701)
[PASS] test_UpdateCredentialSchemaIssuerRegistry() (gas: 29431)
[PASS] test_UpdateOprfKeyRegistry() (gas: 29889)
[PASS] test_UpdateWorldIDRegistry() (gas: 105891)
[PASS] test_UpgradeFailsForNonOwner() (gas: 2546983)
[PASS] test_UpgradeSuccess() (gas: 2614139)
Suite result: ok. 9 passed; 0 failed; 0 skipped; finished in 3.24ms (1.43ms CPU time)

Ran 6 tests for test/core/WorldIDVerifierV2Test.t.sol:WorldIDVerifierV2Test
[PASS] testFuzz_RevertsWhenActionFirstByteNonZero(uint256) (runs: 48, : 37618, ~: 37618)
[PASS] testFuzz_SessionRevertsWhenActionPrefixNot0x02(uint256) (runs: 253, : 37609, ~: 37609)
[PASS] test_PassesActionCheckWhenFirstByteZero() (gas: 352167)
[PASS] test_RevertsWhenActionFirstByteNonZero() (gas: 36994)
[PASS] test_SessionPassesActionCheckWhenFirstByte0x02() (gas: 352377)
[PASS] test_SessionRevertsWhenActionMissing0x02Prefix() (gas: 37196)
Suite result: ok. 6 passed; 0 failed; 0 skipped; finished in 40.69ms (38.98ms CPU time)

Ran 26 tests for test/core/WorldIDRegistryV2.t.sol:WorldIDRegistryV2WIP102Test
[PASS] test_AttackMitigation_AttackerCannotReRecoverAfterVictimRecovery() (gas: 996072)
[PASS] test_AttackMitigation_MaliciousUpdateClearedByRecovery() (gas: 985985)
[PASS] test_CancelRecoveryAgentUpdate_RevertsMethodUnsupported() (gas: 569062)
[PASS] test_ExecuteRecoveryAgentUpdate_RevertsMethodUnsupported() (gas: 568244)
[PASS] test_GetPendingRecoveryAgentUpdate_AfterWindow_ReturnsZero() (gas: 668355)
[PASS] test_GetPendingRecoveryAgentUpdate_DuringWindow_ReturnsTranslatedV2State() (gas: 667546)
[PASS] test_GetPendingRecoveryAgentUpdate_NoActiveUpdate_ReturnsZero() (gas: 569422)
[PASS] test_GetPreviousRecoveryAgentUpdate_AfterWindow_ReturnsZero() (gas: 668653)
[PASS] test_InitiateRecoveryAgentUpdate_RevertsMethodUnsupported() (gas: 571050)
[PASS] test_RecoverAccount_AfterWindow_DoesNotRestorePrev() (gas: 986829)
[PASS] test_RecoverAccount_AfterWindow_PrevAgentFails() (gas: 683491)
[PASS] test_RecoverAccount_AfterWindow_UsesNewAgent() (gas: 983964)
[PASS] test_RecoverAccount_DuringWindow_NewAgentSignatureFails() (gas: 681107)
[PASS] test_RecoverAccount_DuringWindow_UsesPreviousAgent() (gas: 991390)
[PASS] test_RecoverAccount_NoPendingUpdate_UsesCurrentAgent() (gas: 956495)
[PASS] test_RevertRecoveryAgentUpdate_InvalidNonce() (gas: 681774)
[PASS] test_RevertRecoveryAgentUpdate_InvalidSignature() (gas: 681922)
[PASS] test_RevertRecoveryAgentUpdate_RevertsAfterWindowExpired() (gas: 679630)
[PASS] test_RevertRecoveryAgentUpdate_RevertsWhenNoActiveUpdate() (gas: 585217)
[PASS] test_RevertRecoveryAgentUpdate_Success() (gas: 654605)
[PASS] test_UpdateRecoveryAgent_AccountDoesNotExist() (gas: 32036)
[PASS] test_UpdateRecoveryAgent_InvalidNonce() (gas: 597647)
[PASS] test_UpdateRecoveryAgent_InvalidSignature() (gas: 594374)
[PASS] test_UpdateRecoveryAgent_RevertsWhenActiveUpdate() (gas: 676110)
[PASS] test_UpdateRecoveryAgent_SucceedsAfterWindowElapses() (gas: 696442)
[PASS] test_UpdateRecoveryAgent_Success() (gas: 679055)
Suite result: ok. 26 passed; 0 failed; 0 skipped; finished in 174.29ms (168.13ms CPU time)

Ran 1 test for test/core/Verifier.t.sol:VerifierTest
[PASS] testVerifyNullifier() (gas: 295745)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 8.45ms (7.13ms CPU time)

Ran 1 test for test/crosschain/SatelliteVerifierParity.t.sol:SatelliteVerifierParityTest
[PASS] test_keyIdentityMismatch() (gas: 3352448)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 19.08ms (16.94ms CPU time)

Ran 48 tests for test/core/RpRegistry.t.sol:RpRegistryTest
[PASS] testCannotRegisterDuplicateIdInOprfKeyRegistry() (gas: 179218)
[PASS] testCannotRegisterDuplicateRpId() (gas: 151597)
[PASS] testCannotRegisterManyWithInsufficientFee() (gas: 413949)
[PASS] testCannotRegisterManyWithMismatchedArrayLengths() (gas: 29319)
[PASS] testCannotRegisterManyWithMismatchedManagersLength() (gas: 27259)
[PASS] testCannotRegisterManyWithMismatchedSignersLength() (gas: 27678)
[PASS] testCannotRegisterRpWithIdThatConflictsWithCredentialRegistry() (gas: 56347)
[PASS] testCannotRegisterWithInsufficientFee() (gas: 135109)
```

```
[PASS] testCannotRegisterWithInvalidId() (gas: 24492)
[PASS] testCannotRegisterWithZeroAddressManager() (gas: 22034)
[PASS] testCannotRegisterWithZeroAddressSigner() (gas: 22027)
[PASS] testCannotSetFeeRecipientToZeroAddress() (gas: 18854)
[PASS] testCannotSetFeeTokenToZeroAddress() (gas: 19338)
[PASS] testCannotUpdateOprfKeyRegistryToZeroAddress() (gas: 19382)
[PASS] testInitialized() (gas: 18274)
[PASS] testMultipleRpsWithDifferentIds() (gas: 404664)
[PASS] testOnlyOwnerCanSetFeeRecipient() (gas: 31958)
[PASS] testOnlyOwnerCanSetFeeToken() (gas: 574380)
[PASS] testOnlyOwnerCanSetRegistrationFee() (gas: 19443)
[PASS] testOnlyOwnerCanUpdateOprfKeyRegistry() (gas: 133267)
[PASS] testOprfKeyIdMatchesRpId() (gas: 146469)
[PASS] testRegister() (gas: 146459)
[PASS] testRegisterMany() (gas: 388431)
[PASS] testRegisterManyInvalidId() (gas: 33357)
[PASS] testRegisterManyValidatesEachRegistration() (gas: 154271)
[PASS] testRegisterManyValidatesManagerNotZero() (gas: 154025)
[PASS] testRegisterManyValidatesSignerNotZero() (gas: 153842)
[PASS] testRegisterManyWithEmptyArrays() (gas: 15184)
[PASS] testRegisterManyWithFee() (gas: 494558)
[PASS] testRegisterMultipleRps() (gas: 267714)
[PASS] testRegisterWithExcessFee() (gas: 282139)
[PASS] testRegisterWithFee() (gas: 242641)
[PASS] testSetFeeRecipient() (gas: 31861)
[PASS] testSetFeeToken() (gas: 573403)
[PASS] testSetRegistrationFee() (gas: 45241)
[PASS] testUpdateOprfKeyRegistry() (gas: 132363)
[PASS] testUpdateRpCannotReplayOldSignature() (gas: 204169)
[PASS] testUpdateRpInvalidEIP1271Signature() (gas: 421660)
[PASS] testUpdateRpInvalidSignature() (gas: 173465)
[PASS] testUpdateRpManagerTransfer() (gas: 255099)
[PASS] testUpdateRpNonExistentRp() (gas: 32936)
[PASS] testUpdateRpNonceIncrementsOnEachUpdate() (gas: 236972)
[PASS] testUpdateRpNonceMismatch() (gas: 163985)
[PASS] testUpdateRpPartialUpdate() (gas: 203510)
[PASS] testUpdateRpSuccess() (gas: 210265)
[PASS] testUpdateRpToggleActive() (gas: 232744)
[PASS] testUpdateRpWithEIP1271Signature() (gas: 448560)
[PASS] testUpdateRpWithNoUpdate() (gas: 203082)
Suite result: ok. 48 passed; 0 failed; 0 skipped; finished in 21.15ms (18.27ms CPU time)

Ran 13 tests for test/core/WorldIDRegistryV2.t.sol:WorldIDRegistryV2WIP104Test
[PASS] test_CreateAccount_RevertsWithZeroAddress() (gas: 27623)
[PASS] test_InsertAdminAuthenticator() (gas: 936571)
[PASS] test_InsertProvingAuthenticator() (gas: 912533)
[PASS] test_InsertProvingAuthenticator_SkipsAddressValidation() (gas: 1217531)
[PASS] test_RemoveAdminAuthenticator() (gas: 1228817)
[PASS] test_RemoveAdminAuthenticator_RevertsWithZeroAddress() (gas: 950962)
[PASS] test_RemoveLastAdmin_RevertsWhenOnlyProvingRemains() (gas: 928058)
[PASS] test_RemoveLastAdmin_SucceedsWhenNoProvingRemains() (gas: 1228597)
[PASS] test_RemoveProvingAuthenticator() (gas: 1529797)
[PASS] test_RemoveProvingAuthenticator_RevertsWithNonZeroAddress() (gas: 927880)
[PASS] test_SetMaxAuthenticators_RevertsAbove48() (gas: 16988)
[PASS] test_SetMaxAuthenticators_SucceedsAt48() (gas: 23976)
[PASS] test_UpdateAuthenticator_RevertsMethodUnsupported() (gas: 571397)
Suite result: ok. 13 passed; 0 failed; 0 skipped; finished in 88.92ms (114.82ms CPU time)

Ran 6 tests for test/core/GasBenchmarkCold.t.sol:ColdInsertManyBench
[PASS] test_cold_current_insertMany_10() (gas: 1669747)
[PASS] test_cold_current_insertMany_100() (gas: 2385609)
[PASS] test_cold_current_insertMany_1000() (gas: 9606081)
[PASS] test_cold_fs_insertMany_10() (gas: 1387320)
[PASS] test_cold_fs_insertMany_100() (gas: 6072316)
[PASS] test_cold_fs_insertMany_1000() (gas: 53433460)
Suite result: ok. 6 passed; 0 failed; 0 skipped; finished in 278.19ms (364.99ms CPU time)

Ran 5 tests for test/core/WorldIDRegistryUpgrade.t.sol:WorldIDRegistryUpgradeTest
[PASS] test_CannotInitializeTwice() (gas: 17048)
[PASS] test_ImplementationCannotBeInitialized() (gas: 4685855)
[PASS] test_OwnershipTransfer() (gas: 4763530)
[PASS] test_UpgradeFailsForNonOwner() (gas: 4736353)
[PASS] test_UpgradeSuccess() (gas: 5668619)
Suite result: ok. 5 passed; 0 failed; 0 skipped; finished in 16.17ms (10.11ms CPU time)
```

```
Ran 6 tests for test/core/GasBenchmarkCold.t.sol:ColdUpdate1000Bench
[PASS] test_cold_current_update_leaf0_1000tree() (gas: 636620)
[PASS] test_cold_current_update_leaf500_1000tree() (gas: 636478)
[PASS] test_cold_current_update_leaf999_1000tree() (gas: 678421)
[PASS] test_cold_fs_update_leaf0_1000tree() (gas: 430837)
[PASS] test_cold_fs_update_leaf500_1000tree() (gas: 431202)
[PASS] test_cold_fs_update_leaf999_1000tree() (gas: 426954)
Suite result: ok. 6 passed; 0 failed; 0 skipped; finished in 353.51ms (35.12ms CPU time)

Ran 2 tests for test/core/GasBenchmarkCold.t.sol:ColdInsertBench
[PASS] test_cold_current_insert() (gas: 485686)
[PASS] test_cold_fs_insert() (gas: 470914)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 895.48ms (7.99ms CPU time)

Ran 4 tests for test/core/GasBenchmarkCold.t.sol:ColdUpdateBench
[PASS] test_cold_current_update_leaf0() (gas: 674009)
[PASS] test_cold_current_update_leaf50() (gas: 646116)
[PASS] test_cold_fs_update_leaf0() (gas: 425087)
[PASS] test_cold_fs_update_leaf50() (gas: 425359)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 908.71ms (24.00ms CPU time)

Ran 77 tests for test/core/WorldIDRegistry.t.sol:WorldIDRegistryTest
[PASS] test_CancelRecoveryAgentUpdate_InvalidNonce() (gas: 659439)
[PASS] test_CancelRecoveryAgentUpdate_InvalidSignature() (gas: 660168)
[PASS] test_CancelRecoveryAgentUpdate_NoPendingUpdate() (gas: 561637)
[PASS] test_CancelRecoveryAgentUpdate_Success() (gas: 653019)
[PASS] test_CannotCreateAccountWithInsufficientFee() (gas: 6679778)
[PASS] test_CannotCreateManyAccountsWithInsufficientFee() (gas: 6685321)
[PASS] test_CannotRecoverAccountWhichHasNoRecoveryAgent() (gas: 571602)
[PASS] test_CannotRecoverWithPendingRecoveryAgent() (gas: 938460)
[PASS] test_CannotRegisterAuthenticatorAddressThatIsAlreadyInUse() (gas: 554480)
[PASS] test_CannotSetFeeRecipientToZeroAddress() (gas: 6603224)
[PASS] test_CannotSetFeeTokenToZeroAddress() (gas: 6601860)
[PASS] test_CreateAccount() (gas: 547263)
[PASS] test_CreateAccountWithExcessFee() (gas: 7049491)
[PASS] test_CreateAccountWithFee() (gas: 7009169)
[PASS] test_CreateAccountWithNoRecoveryAgent() (gas: 612800)
[PASS] test_CreateManyAccounts() (gas: 10981653)
[PASS] test_CreateManyAccountsWithFee() (gas: 7213893)
[PASS] test_ExecuteRecoveryAgentUpdate_NoPendingUpdate() (gas: 550275)
[PASS] test_ExecuteRecoveryAgentUpdate_StillInCooldown() (gas: 646737)
[PASS] test_GetLatestRoot() (gas: 544955)
[PASS] test_GetMaxAuthenticators() (gas: 27295)
[PASS] test_GetNextLeafIndex() (gas: 543955)
[PASS] test_GetPackedAccountData() (gas: 547509)
[PASS] test_GetPendingRecoveryAgentUpdate_NoPending() (gas: 545479)
[PASS] test_GetPendingRecoveryAgentUpdate_WithPending() (gas: 643034)
[PASS] test_GetProof() (gas: 799952)
[PASS] test_GetProofRevertsForMissingAccount() (gas: 547809)
[PASS] test_GetRecoveryAgent() (gas: 541556)
[PASS] test_GetRecoveryAgentUpdateCooldown_DefaultValue() (gas: 16003)
[PASS] test_GetRecoveryCounter() (gas: 542766)
[PASS] test_GetRootTimestamp() (gas: 24341)
[PASS] test_GetRootValidityWindow() (gas: 26170)
[PASS] test_GetSignatureNonce() (gas: 869389)
[PASS] test_GetTreeDepth() (gas: 16619)
[PASS] test_InitiateRecoveryAgentUpdate_CanOverwritePending() (gas: 670030)
[PASS] test_InitiateRecoveryAgentUpdate_InvalidLeafIndex() (gas: 34629)
[PASS] test_InitiateRecoveryAgentUpdate_InvalidSignature() (gas: 570239)
[PASS] test_InitiateRecoveryAgentUpdate_RevertInvalidNonce() (gas: 572019)
[PASS] test_InitiateRecoveryAgentUpdate_Success() (gas: 651316)
[PASS] test_InsertAuthenticatorDuplicatePubkeyId() (gas: 550078)
[PASS] test_InsertAuthenticatorDuplicatePubkeyIdNonZeroIndex() (gas: 573513)
[PASS] test_InsertAuthenticatorSuccess() (gas: 890351)
[PASS] test_MockERC1271Wallet_Validation() (gas: 269227)
[PASS] test_OnlyOwnerCanSetFeeRecipient() (gas: 6605540)
[PASS] test_OnlyOwnerCanSetFeeToken() (gas: 7146556)
[PASS] test_OnlyOwnerCanSetRegistrationFee() (gas: 6583377)
[PASS] test_RecoverAccountRevertsWithIncorrectOffchainSignerCommitment() (gas: 638699)
[PASS] test_RecoverAccountSuccess() (gas: 1292754)
[PASS] test_RecoverAccountWithERC1271Wallet() (gas: 1164991)
[PASS] test_RecoverAccount_DoesNotApplyPendingUpdate() (gas: 934404)
[PASS] test_RemoveAuthenticatorSuccess() (gas: 871995)
[PASS] test_RemoveAuthenticator_RevertWhenRemovingStaleAuthenticatorAfterRecovery() (gas: 958178)
[PASS] test_RemoveAuthenticator_SucceedsWhenPubkeyIdExceedsLoweredLimit() (gas: 884667)
```

```
[PASS] test_SetFeeRecipient() (gas: 6608269)
[PASS] test_SetFeeToken() (gas: 7151276)
[PASS] test_SetMaxAuthenticators() (gas: 565298)
[PASS] test_SetMaxAuthenticators_RevertWhen_ValueAboveLimit() (gas: 31549)
[PASS] test_SetMaxAuthenticators_SucceedsAtMaxValidValue() (gas: 24059)
[PASS] test_SetMaxAuthenticators_SucceedsBelowMaxValue() (gas: 36687)
[PASS] test_SetRecoveryAgentUpdateCooldown_CanSetToZero() (gas: 615641)
[PASS] test_SetRecoveryAgentUpdateCooldown_OnlyOwner() (gas: 17209)
[PASS] test_SetRecoveryAgentUpdateCooldown_Success() (gas: 29280)
[PASS] test_SetRegistrationFee() (gas: 6607828)
[PASS] test_TreeDepth() (gas: 16375)
[PASS] test_UpdateAuthenticatorInvalidLeafIndex() (gas: 559253)
[PASS] test_UpdateAuthenticatorInvalidNonce() (gas: 576236)
[PASS] test_UpdateAuthenticatorSuccess() (gas: 875729)
[PASS] test_UpdateAuthenticator_SucceedsWhenPubkeyIdExceedsLoweredLimit() (gas: 905809)
[PASS] test_UpdateRecoveryAddressToZeroAddress() (gas: 609391)
[PASS] test_UpdateRecoveryAgentFullFlow() (gas: 625394)
[PASS] test_isValidRoot_customValidityWindow() (gas: 563572)
[PASS] test_isValidRoot_expiresAfterWindow() (gas: 561353)
[PASS] test_isValidRoot_latestRootAlwaysValid() (gas: 32938)
[PASS] test_isValidRoot_unknownRootReturnsFalse() (gas: 18934)
[PASS] test_isValidRoot_zeroWindowMakesHistoricalRootsInvalid() (gas: 561745)
[PASS] test_setRootValidityWindow_emitsEvent() (gas: 30667)
[PASS] test_setRootValidityWindow_onlyOwner() (gas: 17457)
Suite result: ok. 77 passed; 0 failed; 0 skipped; finished in 930.87ms (400.43ms CPU time)

Ran 6 tests for test/core/HashBench.t.sol:LeanIMTTest
[PASS] test_Poseidon2T2() (gas: 7360898)
[PASS] test_Poseidon2T2EqualsReference() (gas: 4743435)
[PASS] test_Poseidon2T2EqualsRust() (gas: 42730)
[PASS] test_Poseidon2T2Reference() (gas: 40115964)
[PASS] test_PoseidonT3() (gas: 42365716)
[PASS] test_PoseidonT4() (gas: 37701478)
Suite result: ok. 6 passed; 0 failed; 0 skipped; finished in 1.17s (1.68s CPU time)

Ran 36 tests for test/core/CredentialSchemaIssuerRegistry.t.sol:CredentialIssuerRegistryTest
[PASS] testCannotRegisterDuplicateIdInCredentialRegistry() (gas: 120435)
[PASS] testCannotRegisterDuplicateIdInOprfKeyRegistry() (gas: 149230)
[PASS] testCannotRegisterWithEmptyPubkey() (gas: 38083)
[PASS] testCannotRegisterWithInsufficientFee() (gas: 132882)
[PASS] testCannotRegisterWithZeroSigner() (gas: 18087)
[PASS] testCannotReplayIssuerSchemaUri() (gas: 281091)
[PASS] testCannotSetFeeRecipientToZeroAddress() (gas: 18921)
[PASS] testCannotSetFeeTokenToZeroAddress() (gas: 19005)
[PASS] testCannotUpdateOprfKeyRegistryToZeroAddress() (gas: 19099)
[PASS] testCannotUpdateSchemaUriAfterRemoval() (gas: 146911)
[PASS] testCannotUpdateSchemaUriForNonExistentIssuer() (gas: 42459)
[PASS] testCannotUpdateSchemaUriToSameSchemaUri() (gas: 253610)
[PASS] testMultipleIssuersWithDifferentIds() (gas: 322284)
[PASS] testOnlyIssuerCanUpdateSchemaUri() (gas: 150637)
[PASS] testOnlyOwnerCanSetFeeRecipient() (gas: 30300)
[PASS] testOnlyOwnerCanSetFeeToken() (gas: 572655)
[PASS] testOnlyOwnerCanSetRegistrationFee() (gas: 17642)
[PASS] testOnlyOwnerCanUpdateOprfKeyRegistry() (gas: 157993)
[PASS] testRegisterAndGetters() (gas: 123942)
[PASS] testRegisterWithExcessFee() (gas: 254114)
[PASS] testRegisterWithFee() (gas: 214419)
[PASS] testRegisterWithZeroFee() (gas: 127757)
[PASS] testRemoveDeletesSchemaUri() (gas: 207943)
[PASS] testRemoveFlow() (gas: 138055)
[PASS] testRemoveWithERC1271Wallet() (gas: 348297)
[PASS] testSetFeeRecipient() (gas: 31735)
[PASS] testSetFeeToken() (gas: 573405)
[PASS] testSetRegistrationFee() (gas: 45003)
[PASS] testUpdateIssuerSchemaUriFlow() (gas: 276917)
[PASS] testUpdateIssuerSchemaUriWithERC1271Wallet() (gas: 492777)
[PASS] testUpdateOprfKeyRegistry() (gas: 160251)
[PASS] testUpdatePubkeyFlow() (gas: 177457)
[PASS] testUpdatePubkeyWithERC1271Wallet() (gas: 427927)
[PASS] testUpdateSignerFlow() (gas: 171468)
[PASS] testUpdateSignerToSameSigner() (gas: 133925)
[PASS] testUpdateSignerWithERC1271Wallet() (gas: 420687)
Suite result: ok. 36 passed; 0 failed; 0 skipped; finished in 2.49s (33.83ms CPU time)

Ran 4 tests for test/core/BinaryIMT.t.sol:BinaryIMTTest
```

```
[PASS] test_InsertManyCorrectness() (gas: 262900215)
[PASS] test_InsertManyTree() (gas: 9606320)
[PASS] test_InsertTree() (gas: 253294935)
[PASS] test_UpdateTree() (gas: 2099304)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 2.55s (5.16s CPU time)

Ran 11 tests for test/core/FullStorageBinaryIMT.t.sol:FullStorageBinaryIMTCorrectnessTest
[PASS] test_getProof_returns_correct_siblings() (gas: 1283140)
[PASS] test_insert_after_update() (gas: 4344938)
[PASS] test_multiple_updates_correctness() (gas: 5623249)
[PASS] test_roots_match_after_100_inserts() (gas: 56210191)
[PASS] test_roots_match_after_10_inserts() (gas: 7402274)
[PASS] test_roots_match_after_single_insert() (gas: 2544255)
[PASS] test_roots_match_insertMany_10() (gas: 5278603)
[PASS] test_roots_match_insertMany_100() (gas: 32731019)
[PASS] test_roots_match_insertMany_1000() (gas: 306795885)
[PASS] test_update_leaf1_matches_reference() (gas: 3832837)
[PASS] test_update_matches_reference() (gas: 3833304)
Suite result: ok. 11 passed; 0 failed; 0 skipped; finished in 2.84s (3.52s CPU time)

Ran 12 tests for test/core/FullStorageBinaryIMT.t.sol:GasBenchmarkFullStorage
[PASS] test_fs_insertMany_10() (gas: 1389117)
[PASS] test_fs_insertMany_100() (gas: 6078687)
[PASS] test_fs_insertMany_1000() (gas: 53485447)
[PASS] test_fs_insert_1000_avg() (gas: 304552250)
[PASS] test_fs_insert_after100() (gas: 29880530)
[PASS] test_fs_insert_first() (gas: 958946)
[PASS] test_fs_sequential_updates_3() (gas: 2246151)
[PASS] test_fs_update_2leaf_tree() (gas: 1469417)
[PASS] test_fs_update_after100_leaf0() (gas: 29814656)
[PASS] test_fs_update_after100_leaf50() (gas: 29814625)
[PASS] test_fs_update_after100_leaf99() (gas: 29814510)
[PASS] test_fs_update_leaf0_singleLeafTree() (gas: 1203244)
Suite result: ok. 12 passed; 0 failed; 0 skipped; finished in 2.87s (4.40s CPU time)

Ran 25 test suites in 2.88s (15.87s CPU time): 326 tests passed, 0 failed, 0 skipped (326 total tests)
```

9 About Nethermind

Nethermind is a Blockchain Research and Software Engineering company. Our work touches every part of the web3 ecosystem - from layer 1 and layer 2 engineering, cryptography research, and security to application-layer protocol development. We offer strategic support to our institutional and enterprise partners across the blockchain, digital assets, and DeFi sectors, guiding them through all stages of the research and development process, from initial concepts to successful implementation.

We offer security audits of projects built on EVM-compatible chains and Starknet. We are active builders of the Starknet ecosystem, delivering a node implementation, a block explorer, a Solidity-to-Cairo transpiler, and formal verification tooling. Nethermind also provides strategic support to our institutional and enterprise partners in blockchain, digital assets, and decentralized finance (DeFi). In the next paragraphs, we introduce the company in more detail.

Blockchain Security: At Nethermind, we believe security is vital to the health and longevity of the entire Web3 ecosystem. We provide security services related to Smart Contract Audits, Formal Verification, and Real-Time Monitoring. Our Security Team comprises blockchain security experts in each field, often collaborating to produce comprehensive and robust security solutions. The team has a strong academic background, can apply state-of-the-art techniques, and is experienced in analyzing cutting-edge Solidity and Cairo smart contracts, such as ArgentX and StarkGate (the bridge connecting Ethereum and StarkNet). Most team members hold a Ph.D. degree and actively participate in the research community, accounting for 240+ articles published and 1,450+ citations in Google Scholar. The security team adopts customer-oriented and interactive processes where clients are involved in all stages of the work.

Blockchain Core Development: Our core engineering team, consisting of over 20 developers, maintains, improves, and upgrades our flagship product - the Nethermind Ethereum Execution Client. The client has been successfully operating for several years, supporting both the Ethereum Mainnet and its testnets, and now accounts for nearly a quarter of all synced Mainnet nodes. Our unwavering commitment to Ethereum's growth and stability extends to sidechains and layer 2 solutions. Notably, we were the sole execution layer client to facilitate Gnosis Chain's Merge, transitioning from Aura to Proof of Stake (PoS), and we are actively developing a full-node client to bolster Starknet's decentralization efforts. Our core team equips partners with tools for seamless node set-up, using generated docker-compose scripts tailored to their chosen execution client and preferred configurations for various network types.

DevOps and Infrastructure Management: Our infrastructure team ensures our partners' systems operate securely, reliably, and efficiently. We provide infrastructure design, deployment, monitoring, maintenance, and troubleshooting support, allowing you to focus on your core business operations. Boasting extensive expertise in Blockchain as a Service, private blockchain implementations, and node management, our infrastructure and DevOps engineers are proficient with major cloud solution providers and can host applications in-house or on clients' premises. Our global in-house SRE teams offer 24/7 monitoring and alerts for both infrastructure and application levels. We manage over 5,000 public and private validators and maintain nodes on major public blockchains such as Polygon, Gnosis, Solana, Cosmos, Near, Avalanche, Polkadot, Aptos, and StarkWare L2. Sedge is an open-source tool developed by our infrastructure experts, designed to simplify the complex process of setting up a proof-of-stake (PoS) network or chain validator. Sedge generates docker-compose scripts for the entire validator set-up based on the chosen client, making the process easier and quicker while following best practices to avoid downtime and being slashed.

Cryptography Research: At Nethermind, our cryptography Research team conducts cutting-edge internal research and collaborates closely with external partners on cryptographic protocols, consensus design, succinct arguments and folding schemes, elliptic curve-based STARK protocols, post-quantum security and zero-knowledge proofs (ZKPs). Our research has led to influential contributions, including Zinc (Crypto '25), Mova, FLI (Asiacrypt '24), and foundational results in Fiat-Shamir security and STARK proof batching. Complementing this theoretical work, our engineering expertise is demonstrated through implementations such as the Latticefold aggregation scheme, the Labrador proof system, zkvm-benchmarks, and Plonk Verifier in Cairo. This combined strength in theory and engineering enables us to deliver cutting-edge cryptographic solutions to partners and clients.

Smart Contract Development & DeFi Research: Our smart contract development and DeFi research team comprises 40+ world-class engineers who collaborate closely with partners to identify needs and work on value-adding projects. The team specializes in Solidity and Cairo development, architecture design, and DeFi solutions, including DEXs, AMMs, structured products, derivatives, and money market protocols, as well as ERC20, 721, and 1155 token design. Our research and data analytics focuses on three key areas: technical due diligence, market research, and DeFi research. Utilizing a data-driven approach, we offer in-depth insights and outlooks on various industry themes.

Our suite of L2 tooling: Warp is Starknet's approach to EVM compatibility. It allows developers to take their Solidity smart contracts and transpile them to Cairo, Starknet's smart contract language. In the short time since its inception, the project has accomplished many achievements, including successfully transpiling Uniswap v3 onto Starknet using Warp.

- **Voyager** is a user-friendly Starknet block explorer that offers comprehensive insights into the Starknet network. With its intuitive interface and powerful features, Voyager allows users to easily search for and examine transactions, addresses, and contract details. As an essential tool for navigating the Starknet ecosystem, Voyager is the go-to solution for users seeking in-depth information and analysis;
- **Horus** is an open-source formal verification tool for StarkNet smart contracts. It simplifies the process of formally verifying Starknet smart contracts, allowing developers to express various assertions about the behavior of their code using a simple assertion language;
- **Juno** is a full-node client implementation for Starknet, drawing on the expertise gained from developing the Nethermind Client. Written in Golang and open-sourced from the outset, Juno verifies the validity of the data received from Starknet by comparing it to proofs retrieved from Ethereum, thus maintaining the integrity and security of the entire ecosystem.

General Advisory to Clients

As auditors, we recommend that any changes or updates made to the audited codebase undergo a re-audit or security review to address potential vulnerabilities or risks introduced by the modifications. By conducting a re-audit or security review of the modified codebase, you can significantly enhance the overall security of your system and reduce the likelihood of exploitation. However, we do not possess the authority or right to impose obligations or restrictions on our clients regarding codebase updates, modifications, or subsequent audits. Accordingly, the decision to seek a re-audit or security review lies solely with you.

Disclaimer

This report is based on the scope of materials and documentation provided by you to Nethermind in order that Nethermind could conduct the security review outlined in **1. Executive Summary** and **2. Audited Files**. The results set out in this report may not be complete nor inclusive of all vulnerabilities. Nethermind has provided the review and this report on an as-is, where-is, and as-available basis. You agree that your access and/or use, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. Blockchain technology remains under development and is subject to unknown risks and flaws. The review does not extend to the compiler layer, or any other areas beyond the programming language, or other programming aspects that could present security risks. This report does not indicate the endorsement of any particular project or team, nor guarantee its security. No third party should rely on this report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. To the fullest extent permitted by law, Nethermind disclaims any liability in connection with this report, its content, and any related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. Nethermind does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party through the product, any open source or third-party software, code, libraries, materials, or information linked to, called by, referenced by or accessible through the report, its content, and the related services and products, any hyperlinked websites, any websites or mobile applications appearing on any advertising, and Nethermind will not be a party to or in any way be responsible for monitoring any transaction between you and any third-party providers of products or services. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.